A

AfiaHealth

Africa's Premier Healthcare Platform

Installation & Setup Guide

Complete guide for developers and DevOps engineers

Vue 3 | Spring Boot 3 | MySQL 8 | Docker

Hospital Search
Location-based discovery

Emergency Finder
24/7 real-time alerts

Appointment Booking
Doctor scheduling

Telemedicine
Video consultations

Admin Dashboard
Full management panel

Multi-language
EN, FR, HA, YO

AfiaHealth Africa Ltd.
contact@afiahealth.africa | https://afiahealth.africa

Version 1.0.0 | May 2025

Table of Contents

1. Project Overview

What is AfiaHealth?

AfiaHealth is a full-stack healthcare discovery platform for Nigeria and Africa. It connects patients with verified hospitals, doctors, pharmacies, and emergency services through a modern mobile-first web application. Backend: Java Spring Boot REST API. Frontend: Vue 3 with Tailwind CSS. Designed to scale across all 36 Nigerian states.

Platform	Web App (Mobile-Responsive PWA)
Backend	Java 17, Spring Boot 3.2, Spring Security + JWT
Frontend	Vue 3, Vite, Tailwind CSS, Pinia, Vue Router
Database	MySQL 8.0
Cache	Redis 7
Maps	Google Maps JavaScript API + Geocoding API
Auth	JWT Access + Refresh tokens, BCrypt (cost 12)
File Storage	AWS S3 (configurable, local fallback)
Containers	Docker + Docker Compose
Languages	English, French, Hausa, Yoruba (vue-i18n)

Core Features

- Hospital search by city, state, specialty, and proximity (geolocation)
- Emergency hospital finder — nearest 24/7 ER sorted by distance
- Doctor directory with specialty, telemedicine, and availability filters
- Appointment booking — in-person, telemedicine, and home-visit types
- Verified patient reviews: Overall, Cleanliness, Staff, Wait Time, Facilities
- AI-powered hospital recommendation based on symptoms
- Ambulance dispatch with real-time GPS tracking reference
- Pharmacy locator with delivery radius and insurance compatibility
- Waiting time crowdsourcing from verified patient check-ins
- Health articles and SEO blog with slugs
- Full Admin Dashboard: hospitals, doctors, reviews, emergency monitoring
- JWT authentication with refresh-token rotation and BCrypt passwords
- Progressive Web App (PWA) with offline support
- Multi-language: English, French, Hausa, Yoruba

2. System Requirements

Development Machine

Requirement	Minimum	Recommended
Java JDK	17+	17 LTS (Temurin)
Maven	3.8+	3.9+
Node.js	18+	20 LTS
npm	9+	10+
MySQL	8.0+	8.0 LTS
Redis	6+	7+ Alpine
Docker	24+ (optional)	25+
Docker Compose	v2+	v2.24+
RAM	8 GB	16 GB
Disk	5 GB	20 GB

Production Server

Requirement	Minimum	Recommended
CPU	2 vCPU	4 vCPU
RAM	4 GB	8 GB
Storage	50 GB SSD	100 GB SSD
OS	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
Database	MySQL 8.0	AWS RDS / PlanetScale
Java Runtime	JRE 17	JRE 17 Alpine
Reverse Proxy	Nginx	Nginx + Certbot

External Services

- Google Maps API Key (Maps JavaScript API + Geocoding API enabled)
- SMTP credentials for email (confirmations, password reset)
- AWS S3 bucket for logo/image uploads (optional — local disk works too)

3. Project Structure

Folder Layout

```
afiahealth/
├── backend/
│   ├── src/main/java/com/afiahealth/
│   │   ├── controller/        REST controllers (Hospital, Doctor, Auth...)
│   │   ├── service/           Business logic layer
│   │   ├── repository/        Spring Data JPA repositories
│   │   ├── entity/            JPA entities / DB model classes
│   │   ├── dto/               Request + Response DTOs
│   │   ├── security/          JWT filter, UserDetailsService
│   │   ├── config/            SecurityConfig, CorsConfig
│   │   ├── exception/         GlobalExceptionHandler
│   │   └── util/              SlugGenerator, GeoUtils
│   ├── src/main/resources/
│   │   ├── application.yml    All app configuration
│   │   └── db/migration/      Flyway SQL migration scripts
│   ├── Dockerfile
│   └── pom.xml
│
├── frontend/
│   ├── src/
│   │   ├── views/
│   │   │   ├── home/          HomePage.vue
│   │   │   ├── hospitals/     HospitalListPage + HospitalDetailPage
│   │   │   ├── doctors/       DoctorListPage + DoctorProfilePage
│   │   │   ├── appointments/  AppointmentPage + BookAppointmentPage
│   │   │   ├── emergency/     EmergencyPage.vue
│   │   │   ├── pharmacy/      PharmacyLocatorPage.vue
│   │   │   ├── blog/          BlogPage + BlogDetailPage
│   │   │   ├── auth/          LoginPage + RegisterPage
│   │   │   ├── dashboard/     PatientDashboard.vue
│   │   │   └── admin/         AdminDashboard.vue
│   │   ├── components/
│   │   │   ├── layout/        AppNavbar.vue, AppFooter.vue
│   │   │   ├── hospital/      HospitalCard.vue, ReviewCard.vue
│   │   │   ├── doctor/        DoctorCard.vue, SchedulePicker.vue
│   │   │   └── common/        LoadingSpinner.vue, StarRating.vue
│   │   ├── store/             Pinia stores: auth.js, hospitals.js
│   │   ├── api/axios.js       Configured Axios client + interceptors
│   │   ├── router/index.js    All Vue Router routes + guards
│   │   ├── locales/           en.json, fr.json, ha.json, yo.json
│   │   └── assets/main.css    Tailwind base + Google Fonts
│   ├── index.html
│   ├── vite.config.js
│   ├── tailwind.config.js
│   ├── nginx.conf             Production Nginx config (SPA + API proxy)
│   └── Dockerfile
```

```
■
■■■ docs/schema.sql            Full MySQL database schema (13 tables)
■■■ docker-compose.yml         Full-stack Docker Compose config
■■■ AfiaHealth_Installation_Guide.pdf
```

4. Quick Start — Docker (Recommended)

No Java or Node.js required. Docker starts every service automatically.

Step 1 — Install Docker Desktop

- Download from https://www.docker.com/get-started
- After install, verify in a terminal:

```
docker --version
docker compose version
```

Step 2 — Unzip the Project

```
unzip afiahealth.zip -d afiahealth
cd afiahealth
```

Step 3 — Configure Environment

```
cp frontend/.env.example frontend/.env

# Edit frontend/.env and set your Google Maps API key:
VITE_GOOGLE_MAPS_API_KEY=AIzaSy_your_key_here
VITE_API_BASE_URL=http://localhost:8080/api/v1
```

Step 4 — Build and Start All Services

```
# Build images and start all containers
docker compose up --build

# Or run in background
docker compose up -d --build

# View live API logs
docker compose logs -f api
```

Step 5 — Access the Application

Service	URL	Notes
Frontend (Vue)	http://localhost	Main web application
API (Spring Boot)	http://localhost:8080	REST API server
Swagger UI	http://localhost:8080/swagger-ui.html	Interactive API docs
MySQL	localhost:3306	User: afiahealth / afiahealth123
Redis	localhost:6379	No password in dev mode

Stop All Services

```
docker compose down          # stop containers
docker compose down -v       # stop + delete all data volumes
```

5. Backend Setup — Spring Boot

Prerequisites

- Java JDK 17 — verify: java -version
- Maven 3.8+ — verify: mvn -version
- MySQL 8.0 running on localhost:3306
- Redis running on localhost:6379

Step 1 — Create MySQL Database

```
mysql -u root -p

CREATE DATABASE afiahealth
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'afiahealth'@'localhost' IDENTIFIED BY 'StrongPass123!';
GRANT ALL PRIVILEGES ON afiahealth.* TO 'afiahealth'@'localhost';
FLUSH PRIVILEGES;
EXIT;

# Import the full schema
mysql -u afiahealth -p afiahealth < docs/schema.sql
```

Step 2 — Edit application.yml

```
# backend/src/main/resources/application.yml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/afiahealth?useSSL=false&serverTimezone=UTC
    username: afiahealth
    password: StrongPass123!

jwt:
  secret: your-random-256-bit-secret-key-change-in-production

google:
  maps:
    api-key: AIzaSy_your_google_maps_api_key
```

Step 3 — Build and Run

```
cd backend

# Compile and download dependencies
mvn clean install -DskipTests

# Run with Maven
mvn spring-boot:run
```

```
# OR: build a runnable JAR
mvn clean package -DskipTests
java -jar target/afiahealth-api-1.0.0.jar
```

Step 4 — Verify and Create Admin

```
# Verify health endpoint
curl http://localhost:8080/actuator/health
# Expected: {"status":"UP"}

# Register first user
curl -X POST http://localhost:8080/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"firstName":"Admin","lastName":"User","email":"admin@afiahealth.africa","password":"Admin@123"}'

# Promote to SUPER_ADMIN in MySQL
mysql -u afiahealth -p afiahealth
UPDATE users SET role="SUPER_ADMIN"
  WHERE email="admin@afiahealth.africa";
```

6. Frontend Setup — Vue 3

Prerequisites

- Node.js 18+ from https://nodejs.org
- npm 9+ (bundled with Node.js)
- Backend API running on port 8080

Step 1 — Go to Frontend Folder

```
cd frontend
```

Step 2 — Create .env File

```
cp .env.example .env

# Edit .env:
VITE_API_BASE_URL=http://localhost:8080/api/v1
VITE_GOOGLE_MAPS_API_KEY=AIzaSy_your_key_here
VITE_APP_NAME=AfiaHealth
```

Step 3 — Install Dependencies

```
npm install
# Installs: Vue 3, Tailwind CSS, Pinia, Vue Router,
# axios, vue-i18n, Chart.js, lucide-vue-next, and more
```

Step 4 — Start Development Server

```
npm run dev
# Output:
#   Local:   http://localhost:5173/
#   Network: http://192.168.x.x:5173/
```

Step 5 — Build for Production

```
npm run build
# Output: frontend/dist/ (optimised static files)

# Preview production build locally:
npm run preview
```

Serve with Nginx (Production)

```
sudo cp -r dist/* /var/www/afiahealth/html/
sudo cp nginx.conf /etc/nginx/sites-available/afiahealth
sudo ln -s /etc/nginx/sites-available/afiahealth /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

7. Database Setup (MySQL 8)

Key Tables

Table	Purpose	Key Columns
users	Patients, doctors, admins	uuid, email, role, is_verified
hospitals	Hospital listings	uuid, name, slug, lat, lng, is_verified
doctors	Doctor profiles	user_id, hospital_id, specialty_id, license_number
appointments	Booking records	reference_number, patient_id, doctor_id, status
reviews	Patient reviews	hospital_id, overall_rating, is_approved
ambulances	Ambulance fleet	vehicle_number, status, current_lat/lng
emergency_requests	Emergency calls	reference_number, status, pickup_lat/lng
pharmacies	Pharmacy listings	name, lat, lng, has_delivery, is_verified
insurance_providers	Insurance data	name, plans JSON, countries_available
articles	Health blog	slug, is_published, author_id
telemedicine_sessions	Video calls	appointment_id, join_url, status
audit_logs	Admin audit trail	user_id, action, entity_type, ip_address

Useful Commands

```
# Import schema
mysql -u afiahealth -p afiahealth < docs/schema.sql

# Show tables
mysql -u afiahealth -p afiahealth -e "SHOW TABLES;"

# Row counts
mysql -u afiahealth -p afiahealth -e "
SELECT 'users' as t,COUNT(*) n FROM users UNION
SELECT 'hospitals'   ,COUNT(*)   FROM hospitals UNION
SELECT 'doctors'     ,COUNT(*)   FROM doctors;"
```

8. Environment Variables Reference

Backend (application.yml / System Env)

Variable	Default	Description
DB_HOST	localhost	MySQL hostname
DB_PORT	3306	MySQL port
DB_NAME	afiahealth	Database name
DB_USERNAME	afiahealth	MySQL user
DB_PASSWORD	(required)	MySQL password
REDIS_HOST	localhost	Redis hostname
REDIS_PORT	6379	Redis port
JWT_SECRET	(required)	Min 256-bit signing key
GOOGLE_MAPS_API_KEY	(required)	Google Maps API key
AWS_ACCESS_KEY	(optional)	S3 file uploads
AWS_SECRET_KEY	(optional)	S3 file uploads
AWS_S3_BUCKET	afiahealth-uploads	S3 bucket name
MAIL_HOST	smtp.gmail.com	SMTP server host
MAIL_USERNAME	(required)	Sender email address
MAIL_PASSWORD	(required)	SMTP / App password
FRONTEND_URL	http://localhost:5173	CORS + email links
SERVER_PORT	8080	API port

Frontend (.env)

Variable	Example	Description
VITE_API_BASE_URL	http://localhost:8080/api/v1	Backend API base URL
VITE_GOOGLE_MAPS_API_KEY	AIzaSy...	Google Maps key
VITE_APP_NAME	AfiaHealth	App name in title/meta
VITE_APP_URL	http://localhost:5173	Frontend base URL

9. API Endpoints Reference

Authentication /api/v1/auth

Method	Endpoint	Auth	Description
POST	/auth/register	No	Register new user account
POST	/auth/login	No	Login — returns JWT access + refresh tokens
POST	/auth/refresh	No	Get new access token using refresh token
POST	/auth/logout	Yes	Revoke current refresh token
GET	/auth/me	Yes	Get authenticated user profile
POST	/auth/forgot-password	No	Send password reset email
POST	/auth/reset-password	No	Reset password with emailed token

Hospitals /api/v1/hospitals

Method	Endpoint	Auth	Description
GET	/hospitals	No	Search (query, city, state, specialty, filters)
GET	/hospitals/:uuid	No	Full hospital detail page data
GET	/hospitals/featured	No	Featured and promoted hospitals
GET	/hospitals/nearby	No	Nearest hospitals by lat/lng + radius
GET	/hospitals/emergency	No	Nearest 24/7 emergency hospitals
GET	/hospitals/:uuid/doctors	No	Paginated doctors at this hospital
GET	/hospitals/:uuid/reviews	No	Approved reviews for hospital
POST	/hospitals/:uuid/reviews	Yes	Submit patient review
GET	/hospitals/:uuid/waiting-time	No	Current wait estimate (minutes)
POST	/hospitals	Admin	Create new hospital listing
PUT	/hospitals/:uuid	HospAdm	Update hospital information
PATCH	/hospitals/:uuid/verify	Admin	Verify a hospital listing

Doctors /api/v1/doctors

Method	Endpoint	Auth	Description
GET	/doctors	No	Search (specialty, city, telemedicine flag)
GET	/doctors/:uuid	No	Full doctor profile
GET	/doctors/:uuid/availability	No	Open slots by date range

Method	Endpoint	Auth	Description
POST	/doctors	Admin	Register new doctor profile
PUT	/doctors/:uuid	Doctor	Update own profile

Appointments /api/v1/appointments

Method	Endpoint	Auth	Description
GET	/appointments	Yes	List patient appointments
POST	/appointments	Yes	Book a new appointment
GET	/appointments/:uuid	Yes	Get single appointment details
PATCH	/appointments/:uuid/cancel	Yes	Cancel an appointment

Emergency /api/v1/emergency

Method	Endpoint	Auth	Description
POST	/emergency/ambulance	No	Request ambulance — returns reference
GET	/emergency/ambulances/nearby	No	Available ambulances near lat/lng
GET	/emergency/requests/:ref	No	Track request by reference code

Pharmacies /api/v1/pharmacies

Method	Endpoint	Auth	Description
GET	/pharmacies	No	Search pharmacies (city, delivery, insurance)
GET	/pharmacies/nearby	No	Nearest pharmacies by lat/lng
GET	/pharmacies/:uuid	No	Pharmacy detail data

Admin /api/v1/admin

Method	Endpoint	Auth	Description
GET	/admin/dashboard	Admin	KPI cards and analytics data
GET	/admin/reviews/pending	Admin	Reviews awaiting moderation
PATCH	/admin/reviews/:id/approve	Admin	Approve and publish review
PATCH	/admin/reviews/:id/reject	Admin	Reject review with reason
GET	/admin/emergency/live	Admin	Live emergency request feed
GET	/admin/users	Admin	Paginated user list with roles

10. Feature Guide

Hospital Search

Search by name/specialty/city. Filters: emergency, telemedicine, 24/7, ambulance, insurance. Sort by rating or distance. Geolocation button auto-fills lat/lng. Results paginated 20/page with infinite scroll on mobile.

Emergency Finder

Browser Geolocation API gets user coordinates, then GET /hospitals/emergency returns hospitals with hasEmergency=true sorted by Haversine distance. Drive time shown via Google Maps Distance Matrix API.

Ambulance Request

POST /emergency/ambulance with patient details and GPS. System finds nearest available ambulance, assigns it, returns a tracking reference. No login required — accessible in a real emergency.

Appointment Booking

Patient browses doctor open slots (from availableDays + availableFrom/To), selects type (in-person/telemedicine/home-visit), and receives email confirmation. Doctor sees it in their dashboard instantly.

Telemedicine

When type is telemedicine, a Jitsi room is auto-created and stored in telemedicine_sessions. Both patient and doctor get a secure join link via email and their dashboard 15 min before the scheduled time.

Verified Reviews

Reviews linked to a confirmed appointment are marked Verified Visit. All reviews require admin approval before appearing publicly. Hospital average_rating updates automatically after each approval.

Multi-language

vue-i18n with 4 locales: EN, FR, Hausa, Yoruba. Language stored in localStorage. Switched via navbar dropdown. API reads X-Language header for translated error messages and email templates.

Admin Dashboard

Full CRUD for hospitals, doctors, users. Review moderation queue with approve/reject. Live emergency monitoring. Analytics charts: appointments by month, hospitals by state. All admin actions written to audit_logs.

11. Cloud Deployment Guide

Option A — Docker on a VPS (Simplest)

```
# 1. Provision Ubuntu 22.04 VPS (min 4 GB RAM)
# 2. Install Docker
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER

# 3. Upload project files
scp -r afiahealth/ ubuntu@YOUR_SERVER_IP:/opt/afiahealth

# 4. Configure env + start
cd /opt/afiahealth
nano docker-compose.yml   # set GOOGLE_MAPS_API_KEY etc.
docker compose up -d --build

# 5. SSL certificate
sudo apt install -y nginx certbot python3-certbot-nginx
sudo certbot --nginx -d yourdomain.africa
```

Option B — Managed Services (Scalable)

Component	Service	Notes
Frontend (Vue)	Vercel / Netlify / S3+CloudFront	Deploy dist/ folder
Backend API	AWS Elastic Beanstalk / Railway	Deploy JAR or Docker image
MySQL	AWS RDS / PlanetScale	Enable automated backups
Redis	AWS ElastiCache / Redis Cloud	Enable TLS in production
File Storage	AWS S3	Set bucket CORS for upload origins
Domain + SSL	Cloudflare + Certbot	Free SSL certificates
Monitoring	CloudWatch / Grafana	JVM + DB performance alerts

12. Security Best Practices

JWT Secrets

Use a minimum 256-bit random key. Never commit to Git. Use environment variables or a secrets manager (AWS Secrets Manager / Vault).

HTTPS Only

Terminate SSL at Nginx in production. Use Certbot for free Let's Encrypt certificates. Redirect all HTTP traffic to HTTPS permanently.

Database

Create a dedicated MySQL user with minimum privileges. Enable SSL for DB connections. Disable remote root login. Use managed DB with backups.

Rate Limiting

Limit /login, /register, /forgot-password to prevent brute force. Use Bucket4j (Spring Boot) or Nginx limit_req_zone.

Input Validation

All DTOs use Bean Validation (@NotBlank, @Email, @Size). GlobalExceptionHandler returns clean error responses — no stack traces exposed.

CORS

SecurityConfig.java restricts origins to FRONTEND_URL only in production. Change allowedOriginPatterns from wildcard to your exact domain before go-live.

File Uploads

Whitelist MIME types (image/jpeg, image/png, application/pdf). Enforce 10 MB limit. Store in S3 with random UUID filenames — not on the server.

SQL Injection

Spring Data JPA uses parameterized queries everywhere. Never concatenate raw SQL strings. All repos use JPQL named parameters.

Passwords

BCrypt with cost factor 12. Never return password_hash in API responses. DTOs explicitly choose which fields are returned.

Audit Logging

admin_logs table records all admin actions with user ID, IP, and before/after values. Review weekly; alert on unusual patterns.

13. Troubleshooting

Cannot connect to MySQL

- Verify MySQL is running: sudo systemctl status mysql
- Credentials in application.yml must match the DB user you created
- Confirm the afiahealth database exists: SHOW DATABASES; in MySQL shell
- Using Docker? Allow 30 s for the MySQL healthcheck before the API starts

JWT / 401 Unauthorized errors

- Ensure JWT_SECRET is identical on all API instances
- Authorization header format: Bearer <token> (capital B, one space)
- Access token may be expired (1 h TTL) — call POST /auth/refresh

Frontend shows blank page

- Open DevTools (F12) and check the Console tab for errors
- Verify VITE_API_BASE_URL in .env points to the running backend
- Production Nginx: ensure try_files $uri /index.html is in nginx.conf

Google Maps not loading

- Verify Maps JavaScript API and Geocoding API are enabled in Google Cloud Console
- Check the key has no domain restrictions blocking localhost
- Look for ApiNotActivatedMapError in the browser console

Docker build fails

- Ensure Docker Desktop is running with at least 4 GB RAM allocated
- Run docker system prune to clear old cache, then retry
- Check internet access — Maven/npm must download dependencies on first build

Emails not sending

- For Gmail: enable 2FA and generate an App Password (not your main password)
- Verify MAIL_HOST (smtp.gmail.com), MAIL_PORT (587), USERNAME, and PASSWORD
- Check firewall: outbound port 587 (STARTTLS) must be open on your server

---

Need Help?

Contact support@afiahealth.africa or open a GitHub issue. Include your OS, Java version, Node.js version, and the full error message.